



Versionamento de arquivos.

Imagens de: <http://git-scm.com/book/en/>

"FINAL".doc



FINAL.doc!



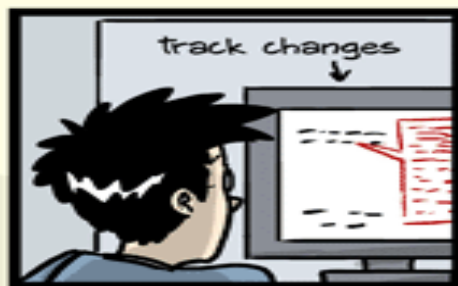
FINAL_rev.2.doc



FINAL_rev.6.COMMENTS.doc



FINAL_rev.8.comments5.
CORRECTIONS.doc



FINAL_rev.18.comments7.
corrections9.MORE.30.doc



FINAL_rev.22.comments49.
corrections.10.##\$%WHYDID
ICOMETOGRADSCHOOL?????.doc



"FINAL".doc

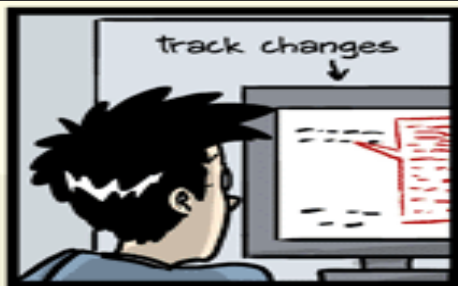


FINAL.doc!



FINAL_rev.2.doc

Você vai passar por isso



FINAL_rev.18.comments7.
corrections9.MORE.30.doc



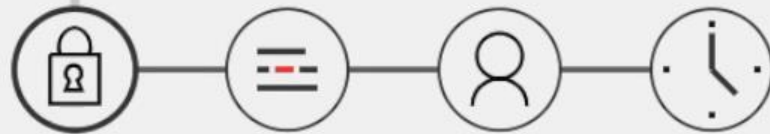
FINAL_rev.22.comments49.
corrections.10. #@\$%WHYDID
ICOMETOGRADSCHOOL?????.doc



O que ocorre no trabalho compartilhado

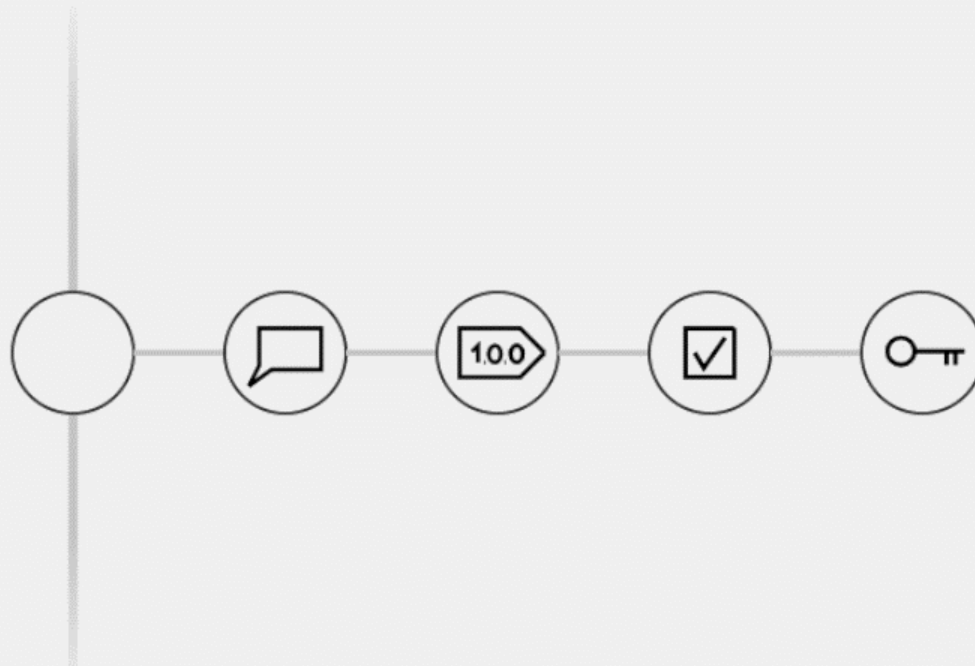
Enquanto um está editando os demais não podem editar o documento original

O que ocorre no trabalho compartilhado



As observações só são enviadas quando o documento está totalmente pronto.

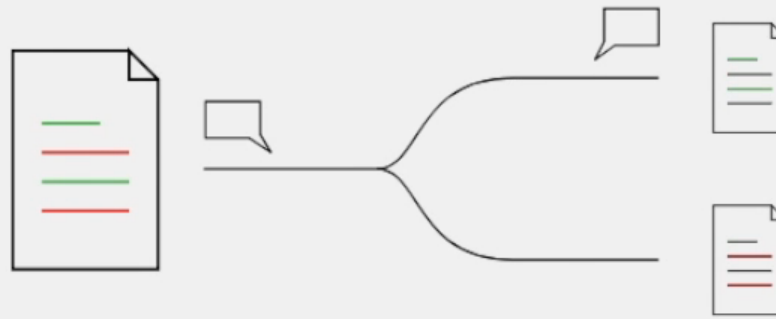
O que ocorre no trabalho compartilhado



O controle de versão é feito com múltiplas cópias do mesmo arquivo

As cópias podem levar a diferentes discussões que podem não ser compartilhadas com os demais autores e revisores.

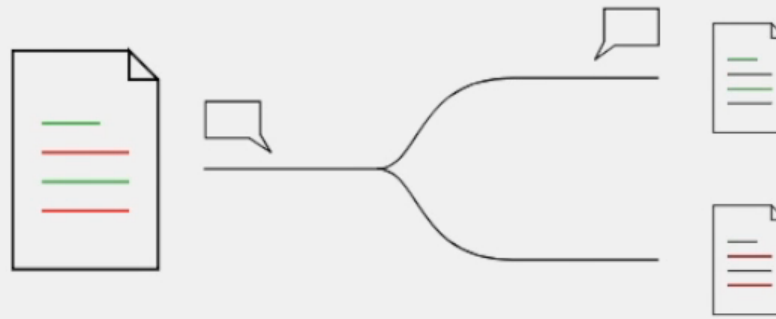
O que ocorre no trabalho compartilhado



O controle de versão é feito com múltiplas cópias do mesmo arquivo

As cópias podem levar a diferentes discussões que podem não ser compartilhadas com os demais autores e revisores.

O que ocorre no trabalho compartilhado

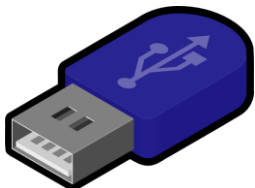


O controle de versão é feito com múltiplas cópias do mesmo arquivo

As cópias podem levar a diferentes discussões que podem não ser compartilhadas com os demais autores e revisores.

Como compartilhar arquivos

Como viabilizar o trabalho em equipe?



Como compartilhar arquivos

Como viabilizar o trabalho em equipe?



O que é controle de versão

- **Caso:** Aluno(s) e professor(es) estão trabalhando em um documento juntos
- **Questão:** Por que não Google Drive ou One Drive.

	Versionamento	Arquivos compartilhados Google/One Drive
Colaboração	Sim	Sim
Versionamento	Sim	Parcial
Rolling Back	Sim	Parcial
Rastreabilidade	Sim	Sim
Disponibilidade	Sim	Sim
Backup	Parcial	Sim

São conceitos diferentes: Gerenciamento de conteúdo vs Gerenciamento de arquivos.

—

Deixar que um programa lide com o controle de versão do seu projeto permite que você foque no seu projeto.

Como resolver as múltiplas versões de arquivos

- Fazendo o *versionamento* através de software específico:
- Um **sistema de controle de versões** (ou *versionamento*), é um software que tem a finalidade de gerenciar diferentes versões no desenvolvimento de um documento qualquer.
 - Esses sistemas são comumente utilizados no desenvolvimento de software para controlar as diferentes versões — histórico e desenvolvimento — dos códigos-fontes e também da documentação.
- A eficácia do controle de versão de software é comprovada por fazer parte das exigências para melhorias do processo de desenvolvimento de certificações tais como CMMI e SPICE

Vantagens do versionamento

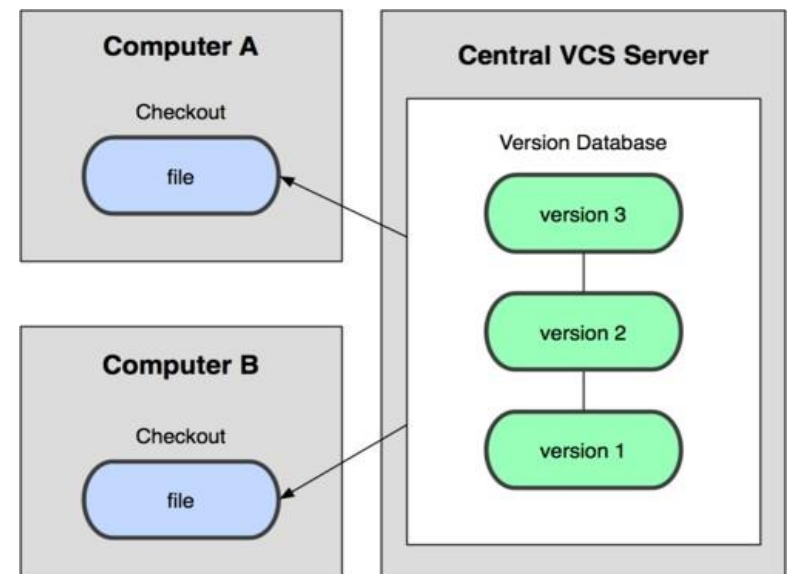
- **Controle do histórico:** facilidade em *desfazer* e possibilidade de analisar o histórico do desenvolvimento, como também facilidade no resgate de versões mais antigas e estáveis. A maioria das implementações permitem analisar as alterações com detalhes, desde a primeira versão até a última.
- **Trabalho em equipe:** um *sistema de controle de versão* permite que diversas pessoas trabalhem sobre o mesmo conjunto de documentos ao mesmo tempo e minimiza o desgaste provocado por problemas com conflitos de edições. É possível que a implementação também tenha um controle sofisticado de acesso para cada usuário ou grupo de usuários.
- **Marcação e resgate de versões estáveis:** a maioria dos sistemas permite marcar onde é que o documento estava com uma versão estável, podendo ser facilmente resgatado no futuro.
- **Ramificação de projeto:** a maioria das implementações possibilita a divisão do projeto em várias linhas de desenvolvimento, que podem ser trabalhadas paralelamente, sem que uma interfira na outra.

Vantagens do versionamento

- **Segurança:** Cada software de controle de versão usa mecanismo para evitar qualquer tipo de invasão de agentes infecciosos nos arquivos. Além do mais, somente usuários com permissão poderão mexer no código.
- **Rastreabilidade:** com a necessidade de sabermos o local, o estado e a qualidade de um arquivo; o controle de versão trás todos esses requisitos de forma que o usuário possa se embasar do arquivo que deseja utilizar.
- **Organização:** Com o software é disponibilizado interface visual que pode ser visto todos arquivos controlados, desde a origem até o projeto por completo.
- **Confiança:** O uso de repositórios remotos ajuda a não perder arquivos por eventos imponderáveis. Além disso é disponível fazer novos projetos sem danificar o desenvolvimento.

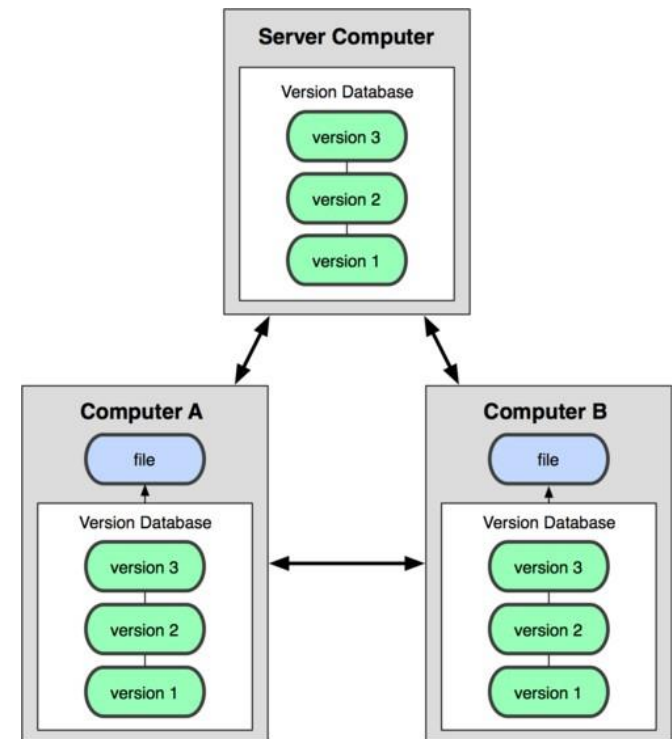
Versionamento centralizado

- No Subversion, CVS, Perforce, etc. Um repositório de servidor central (repo) contém a "cópia oficial" do código
 - o servidor mantém o histórico da versão única do repo
- Você faz "checkouts" dele para sua cópia local
 - você faz modificações locais
 - suas alterações não são versionadas
- Quando estiver pronto, você "faz check-in" de volta ao servidor
 - seu checkin incrementa a versão do repo

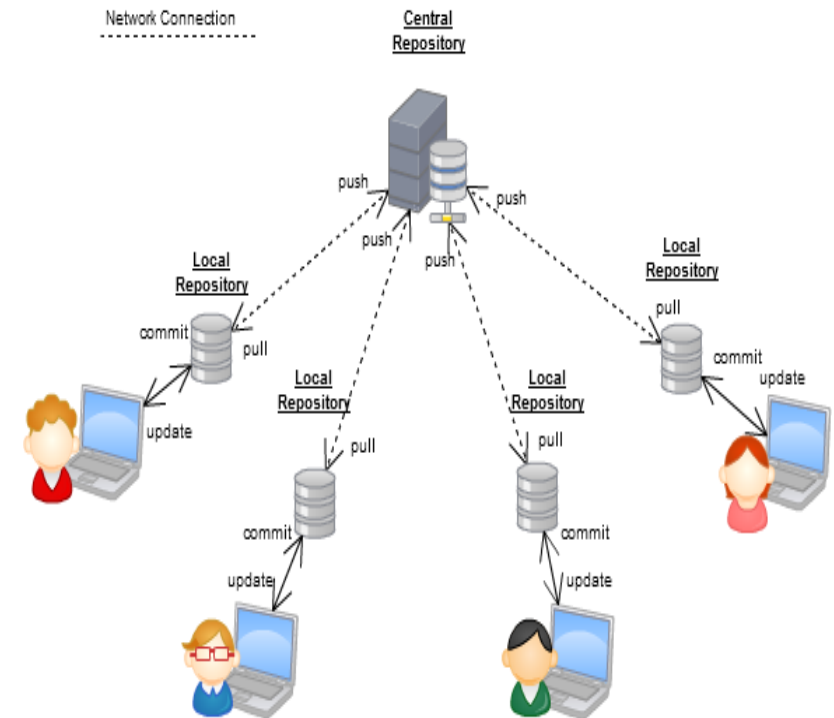
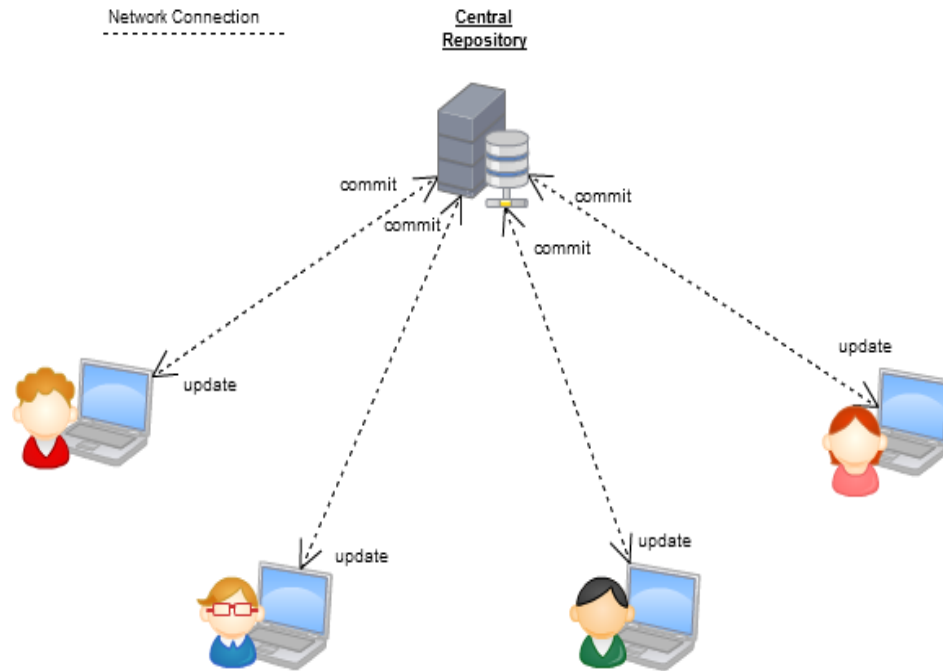


VCS Distribuido(Git)

- No git, mercurial, etc., você não faz "checkout" para um repositório central
 - Você clona(clone) e então (envia)"pull" suas modificações
- Seu repositório local é uma cópia completa de tudo no servidor remoto
- Muitas operações são locais:
 - check in/out no repositório *local*
 - commit mudanças para o *repositorio*
 - repo local mantém o histórico de versões
- Quando estiver pronto, você pode enviar "pull" as alterações de volta ao servidor



Central vs distribuido



Ferramentas de versionamento

- Ajuda a rastrear, gerenciar e distribuir versões
- Em programação é extremamente usual
- Exemplos:

Antigo

Revision Control System (RCS) - 1982

Concurrent Versions System (CVS) - 1986

Subversion (SVN) - 2000

Git - 2005

Novo

Foco

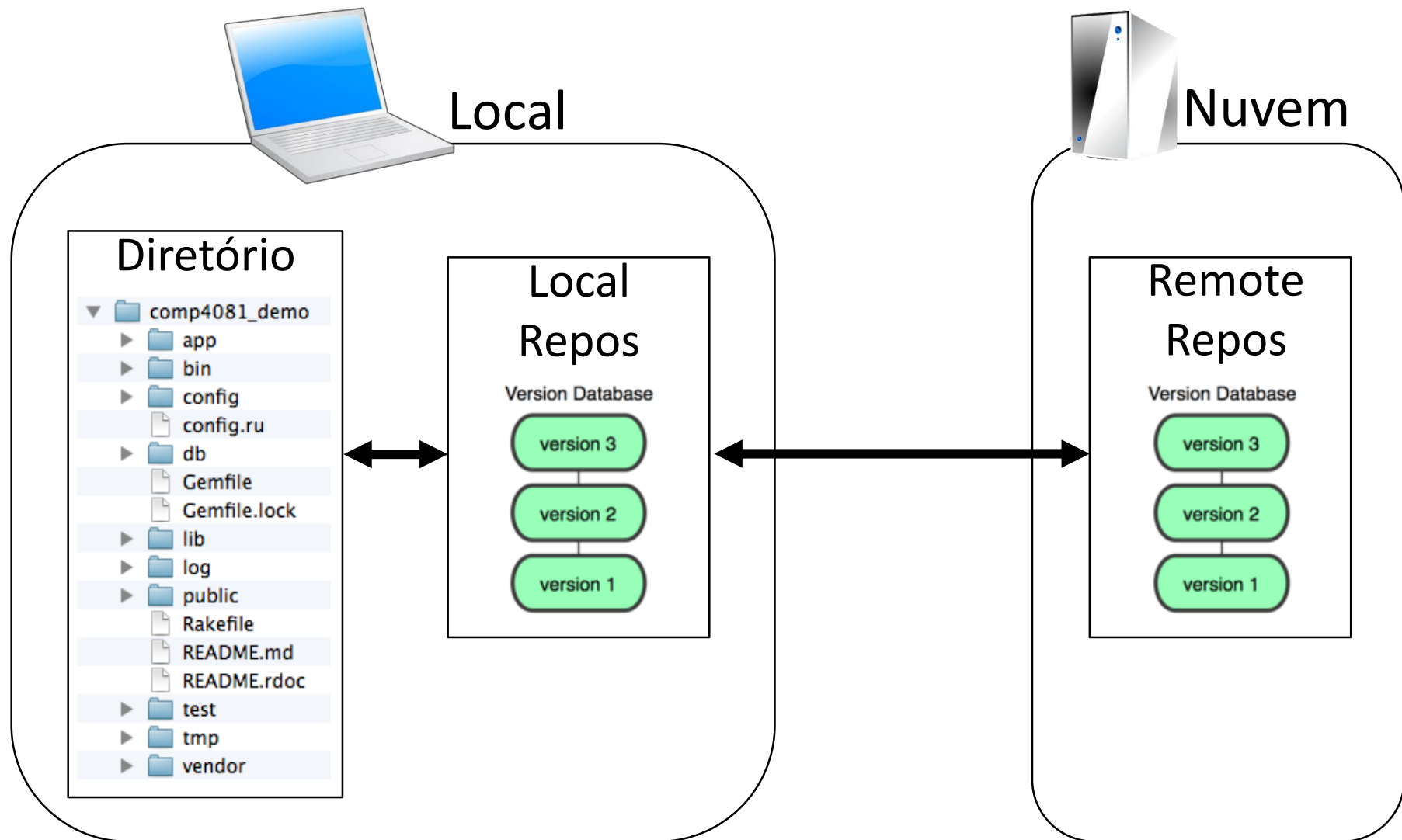


Git

- Criado por Linus Torvalds, criador do Linux, in 2005
 - Originou-se da comunidade de desenvolvimento Linux
 - Projetado para fazer controle de versão no kernel do Linux
- Objetivos do Git:
 - Velocidade
 - Suporte para desenvolvimento não linear (milhares de ramificações paralelas)
 - Totalmente distribuído
 - Capaz de lidar eficientemente com grandes projetos
 - *(Um "git" é um velho rabugento. Linus se referia a si mesmo.)*



Perspectiva



Instalando/Aprendendo

- Git website: <http://git-scm.com/>
 - Livro grátis: <http://git-scm.com/book>
 - Página de referência: <http://gitref.org/index.html>
 - Git tutorial: <http://schacon.github.com/git/gittutorial.html>
 - Git para computação:
 - <http://eagain.net/articles/git-for-computer-scientists/>
- Na linha de comando: (*onde termo = config, add, commit, etc.*)
 - `git help termo`

Git Tools



Windows Git Clients:

- [Sourcetree](#)
- [GitHub for Windows](#) .
- [Tortoise Git](#)

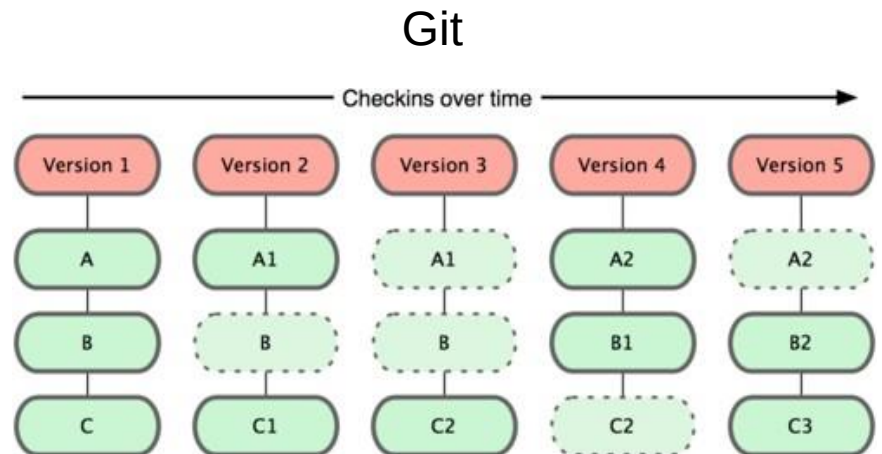
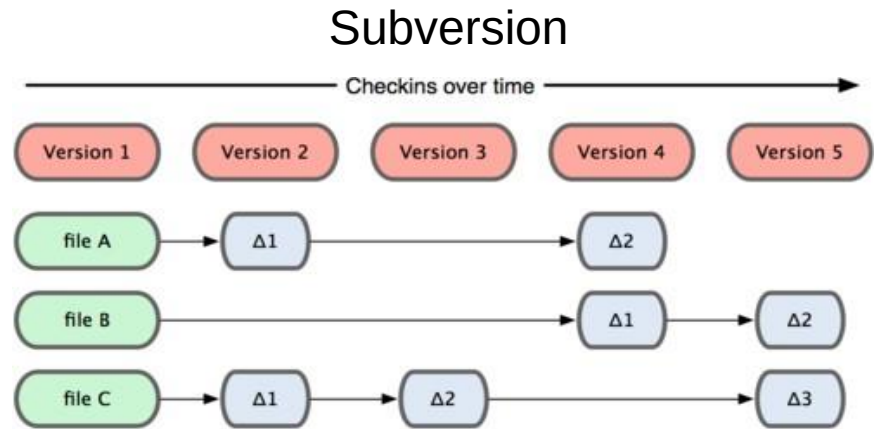
Mac Git Clients:

- [GitUp](#)
- [GitBox](#)
- [Git-Xdev](#)



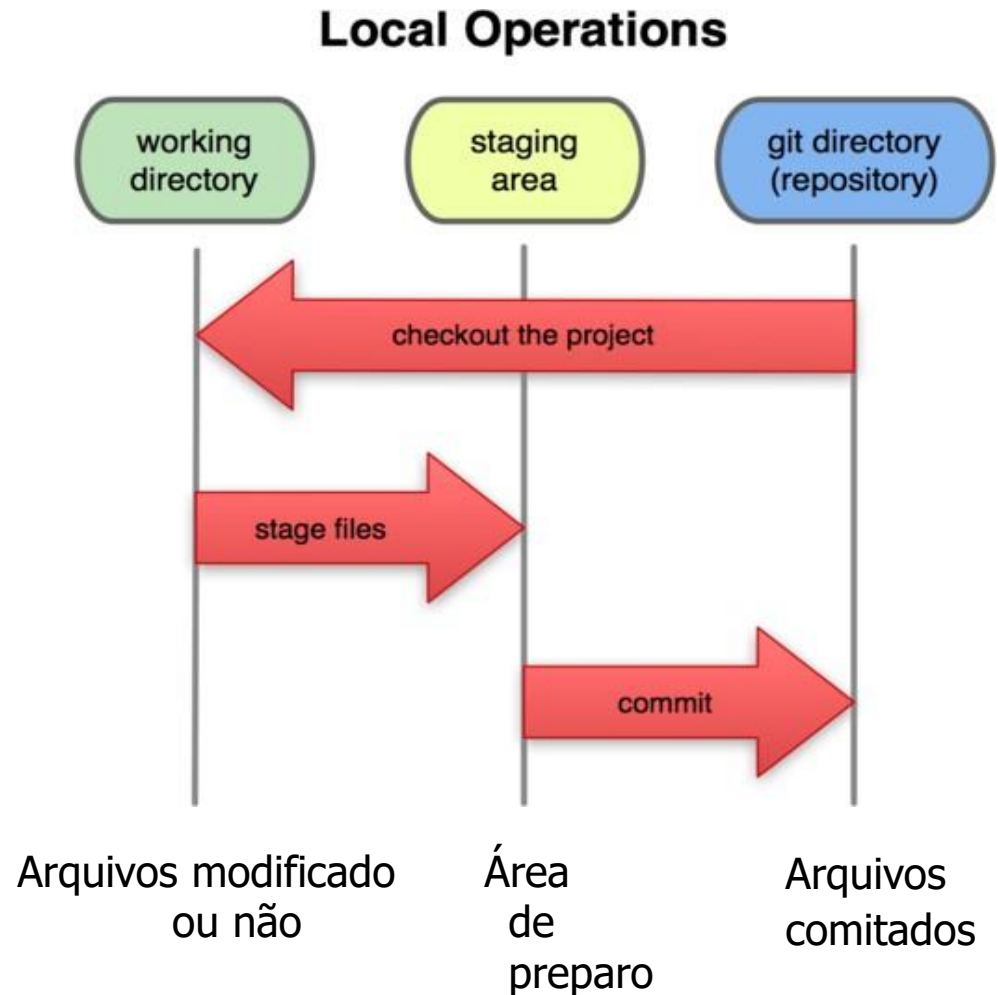
Git snapshots

- O VCS centralizado, como o Subversion, rastreia os dados da versão em cada arquivo individual.
- O Git mantém "instantâneos" de todo o estado do projeto.
 - Cada versão de checkin do código geral tem uma cópia de cada arquivo.
 - Alguns arquivos são alterados em um determinado check-in, outros não.
 - Mais redundância, mas mais rápida.



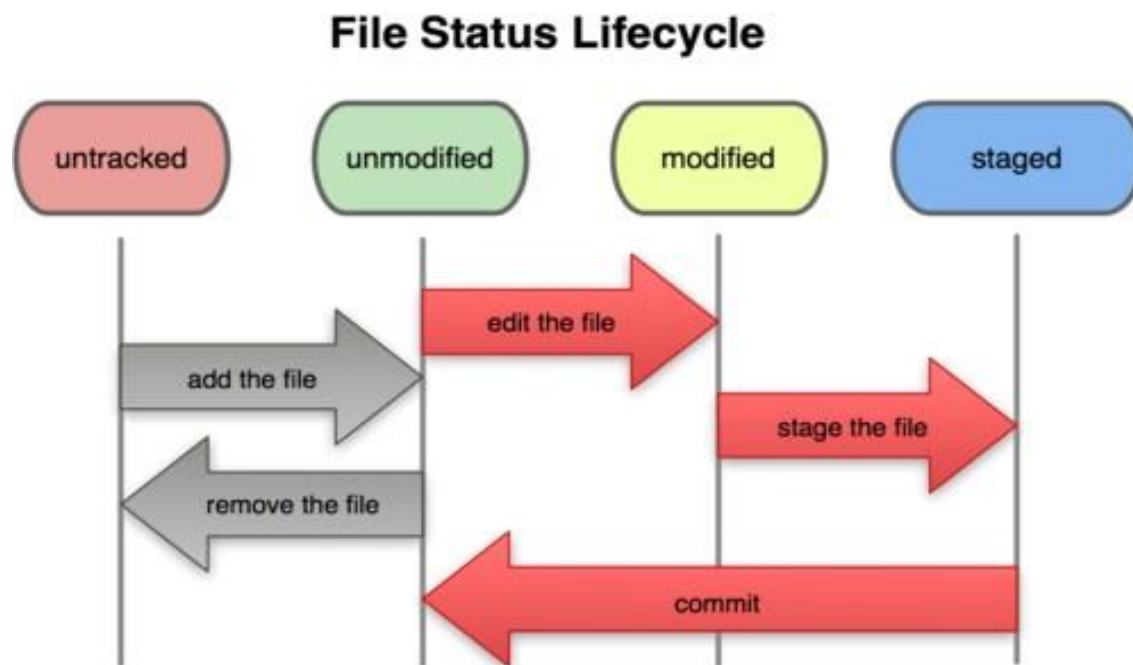
Área local do GIT

- Na sua cópia local no git, os arquivos podem ser:
 - Atualizados no repositório
 - (committed)
 - Verificado e modificado, mas ainda não confirmado
 - (working copy)
 - Ou mantido em uma **"área de preparo"**
 - Staged files estão prontos para serem "comitados"
 - Um commit salva um instantâneo de todo o estado preparado.



Funcionamento do git

- **Modifique** os arquivos em seu diretório de trabalho.
- **Prepare**, adicionando instantâneos deles à sua área de preparação.
- **Commit**, que pega os arquivos na área de preparação e armazena esse instantâneo permanentemente no seu diretório Git.



Git commit checksums

- In Subversion each modification to the central repo increments the version # of the overall repo.
 - In Git, each user has their own copy of the repo, and commits changes to their local copy of the repo before pushing to the central server.
 - So Git generates a unique **SHA-1 hash** (40 character string of hex digits) for every commit.
 - Refers to commits by this ID rather than a version number.
- Often we only see the first 7 characters:
 - 1677b2d Edited first line of readme
 - 258efa7 Added line to readme
 - 0e52da7 Initial commit

Git commit checksums

- No Subversion, cada modificação do repositório central incrementa a versão # do repositório geral.
 - No Git, cada usuário tem sua própria cópia do repositório e confirma as alterações em sua cópia local do repositório antes de enviar para o servidor central.
 - Assim, o Git gera um hash SHA-1 exclusivo (40 caracteres de dígitos hexadecimais) para cada confirmação.
 - Refere-se a commits por este ID em vez de um número de versão.
- Muitas vezes vemos apenas os primeiros 7 caracteres:
 - `1677b2d Edited first line of readme`
 - `258efa7 Added line to readme`
 - `0e52da7 Initial commit`

Configuração inicial do GIT

- Ajuste o nome e e-mail para commit:
 - `git config --global user.name "Bugs Bunny"`
 - `git config --global user.email bugs@gmail.com`
 - You can call `git config -list` to verify these are set.
- Ajuste o editor para configurações (dispensável em muitos casos):
 - `git config --global core.editor nano`
 - (o padrão é vim no Linux, notepad no Windows e text no Mac)

Criando um repositório

Há dois cenários:

- Criando um repositório local na máquina:

- `git init`
 - Isso irá criar um diretório oculto `.git` no diretório que estejas.
 - Então pode-se adicionar arquivos com o comando.
- `git add filename`
- `git commit -m "commit message"`

- **Clonando um repositório remoto** para a pasta atual:

- `git clone url localDirectoryName`
 - Isso criará o diretório local fornecido, contendo uma cópia de trabalho dos arquivos do repositório e um diretório `.git`

Comandos GIT

command	description
git clone <i>url</i> [<i>dir</i>]	Copia um repositório para o Sistema de arquivos.
git add <i>file</i>	Adiciona arquivos a área de preparo
git commit	Grava um snapshot do projeto
git status	Avalia o status dos arquivos na área de preparo e pasta do projeto.
git diff	Mostra as diferenças entre arquivos locais e do repositório ou entre snapshots
git help [<i>command</i>]	Mais informações sobre um comando específico
git pull	buscar de um repositório remoto e tentar se fundir ao ramo atual
git push	Envia suas novas ramificações e dados para um repositório remoto
outros: init, reset, branch, checkout, merge, log, tag	

Adicionando e commit de arquivos

- Quando as alterações são concluídas, o novo código é enviado.
- Um **commit** consiste em um conjunto de arquivos **check-in** e o **diff** entre as versões nova e primária de cada arquivo.
- Cada **check-in** é acompanhado por um nome de usuário e outros metadados.
- Os **check-ins** podem ser exportados do sistema de controle de versão na forma de um patch.

Adicionando e commit de arquivos

- Na primeira vez que solicitamos que um arquivo seja rastreado, e sempre antes de enviarmos um arquivo, devemos adicioná-lo à área de preparação:
 - `git add Hello.java Goodbye.java`
 - Obtém um instantâneo desses arquivos e os adiciona à área de preparação.
 - Em VCS antigo, "add" significa "iniciar o rastreamento deste arquivo". No Git, "add" significa "adicionar na área de preparação", assim ele fará parte do commit.
- Para enviar os arquivos preparados para o repositório:
 - `git commit -m "Fixing bug #22"`
- Para desfazer as alterações em um arquivo antes de você tê-lo confirmado:
 - `git reset HEAD -- filename`
 - `git checkout -- filename`
 - Todos esses comandos estão agindo em sua versão local do repositório.

Ver e desfaz alterações

- Para ver o estado dos arquivos locais e na área de preparação:
 - `git status` ou `git status -s` (versão curta)
- Para ver as modificações dos arquivos:
 - `git diff`
- Para ver a diferença dos arquivos a na área de preparação:
 - `git diff --cached`
- Para ver a lista de modificações feitas no repositório:
 - `git log` or `git log --oneline` (shorter version)
 - 1677b2d Edited first line of readme
 - 258efa7 Added line to readme
 - 0e52da7 Initial commit
 - `git log -5` (to show only the 5 most recent updates), etc.

Um exemplo

```
[rea@attu1 superstar]$ emacs rea.txt
[rea@attu1 superstar]$ git status
  no changes added to commit
  (use "git add" and/or "git commit -a")
[rea@attu1 superstar]$ git status -s
  M rea.txt
[rea@attu1 superstar]$ git diff
  diff --git a/rea.txt b/rea.txt
[rea@attu1 superstar]$ git add rea.txt
[rea@attu1 superstar]$ git status
  #       modified:   rea.txt
[rea@attu1 superstar]$ git diff --cached
  diff --git a/rea.txt b/rea.txt
[rea@attu1 superstar]$ git commit -m "Created new text file"
```

Branching e merging

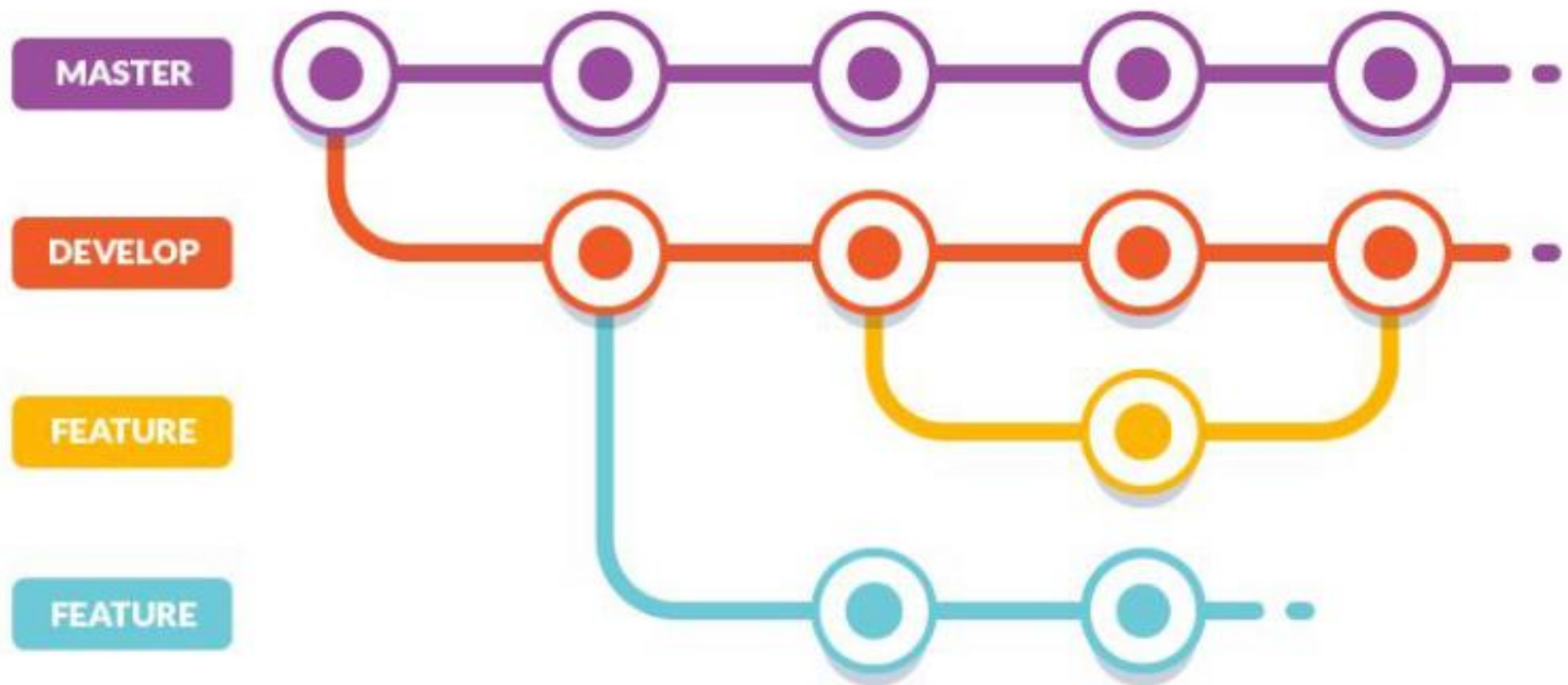
- Ramificações() permite várias cópias da base de código dentro de um único repositório.
 - Clientes diferentes têm requisitos diferentes
 - O cliente A quer os recursos A, B, C
 - O cliente B quer os recursos A e C, mas não B, porque seu computador é antigo e fica muito lento.
 - O cliente C quer apenas o recurso A devido aos custos
 - Cada cliente tem seu próprio ramo.
- Diferentes partes do documento podem servir a públicos diferentes.

Branching e merging

O Git usa ramificações pesadamente para alternar entre várias tarefas.

- Para criar um ramo
 - `git branch name`
- Para listar os ramos: (* = ramo atual)
 - `git branch`
- Para mudar de ramo:
 - `git checkout branchname`
- Para mesclar alterações de uma ramificação no mestre local:
 - `git checkout master`
 - `git merge branchname`

Branching e merging



Na vida real pode ser mais complicado



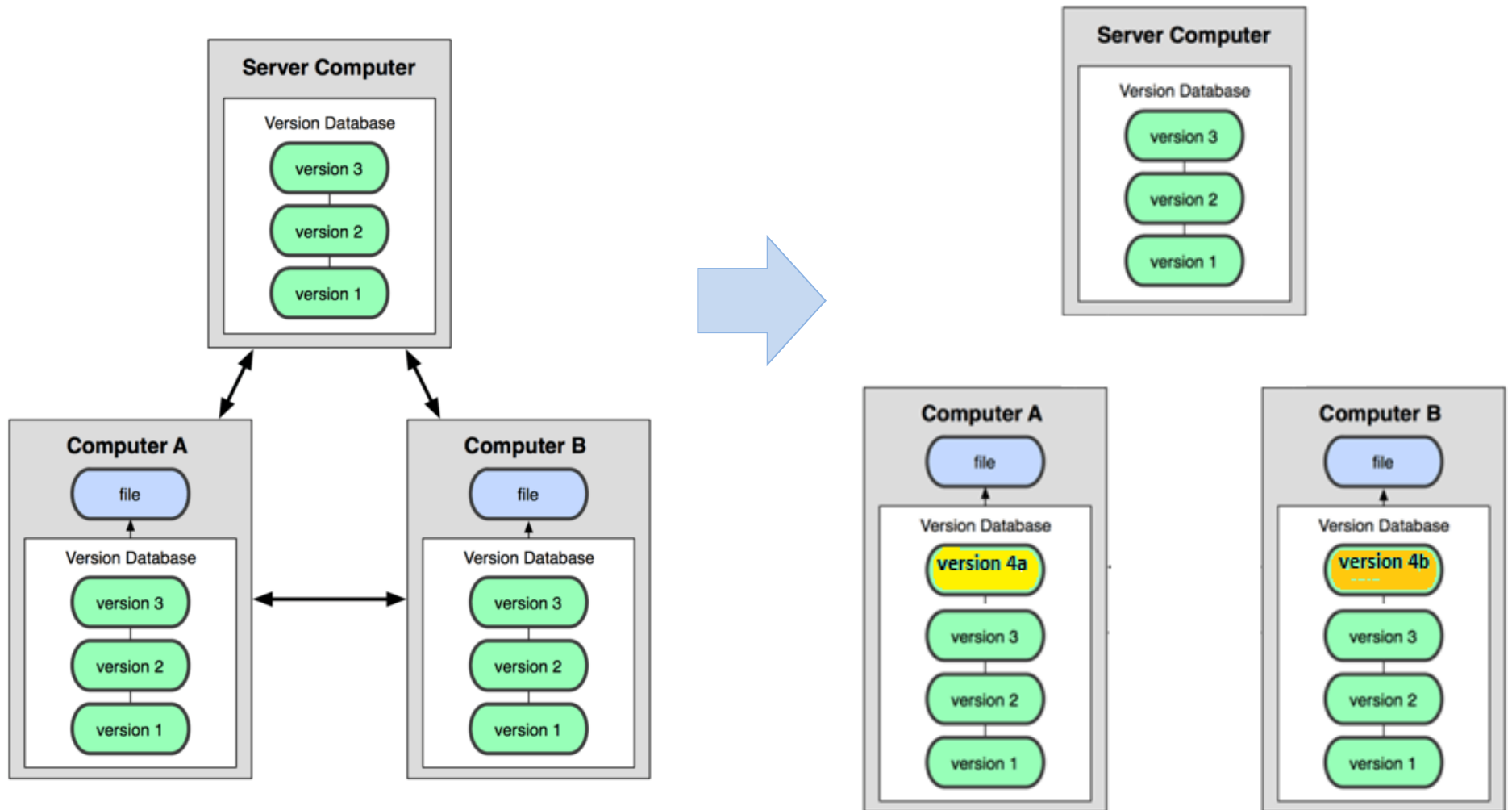
Ramificações do kernel do Linux

Mesclando conflitos

- Há ocasiões em que várias versões de um arquivo precisam ser recolhidas em uma única versão.
- Por exemplo. Um recurso de um ramo é necessário em outro
- Esse processo é conhecido como uma mesclagem.
- Difícil e perigoso para fazer no CVS
- Fácil e barato para fazer isso git



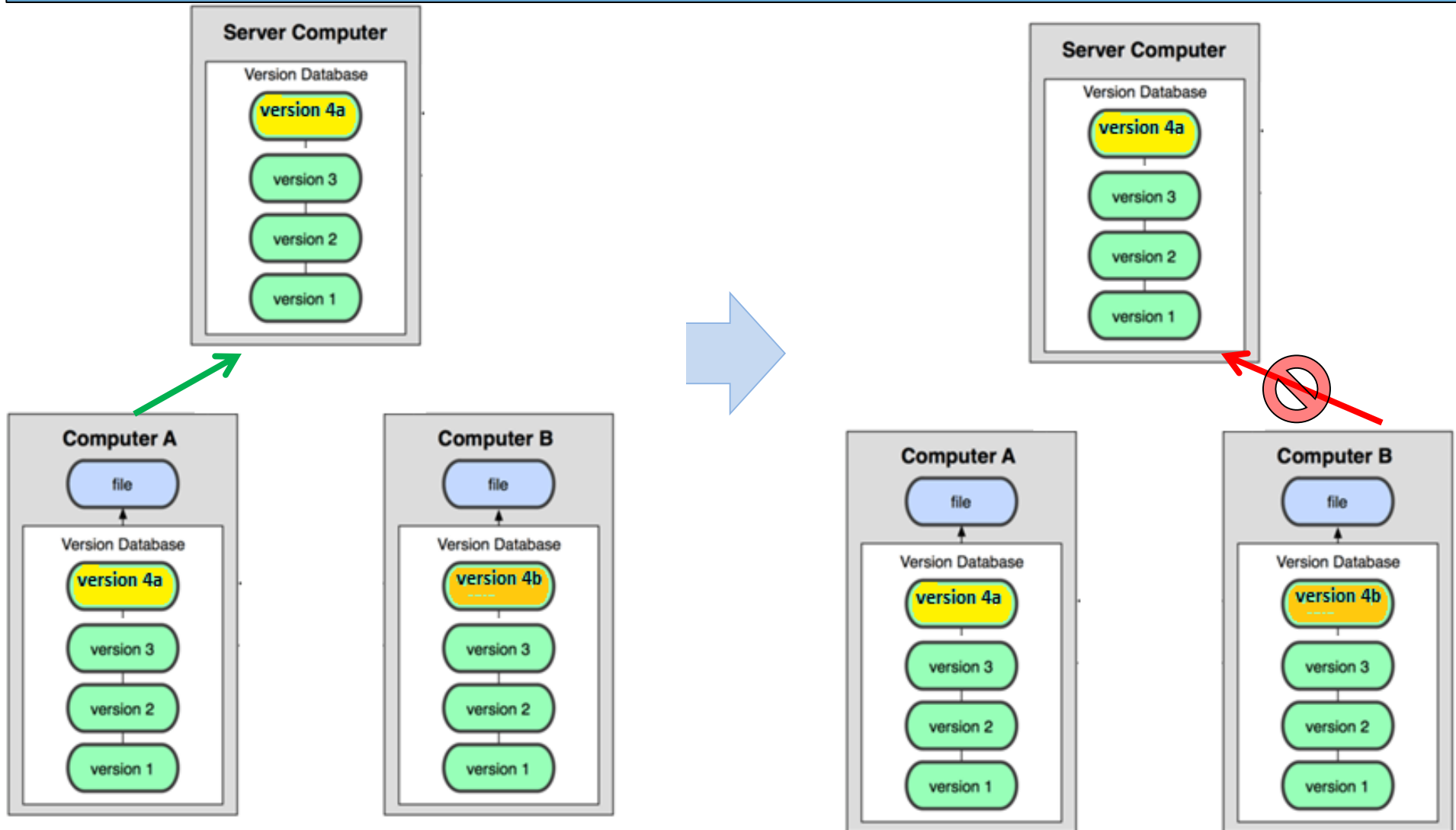
Mesclando conflitos Ex1



1. Ambos possuem os mesmos arquivos

2. Ambos editam os mesmos arquivos ao mesmo tempo modificando as mesmas partes

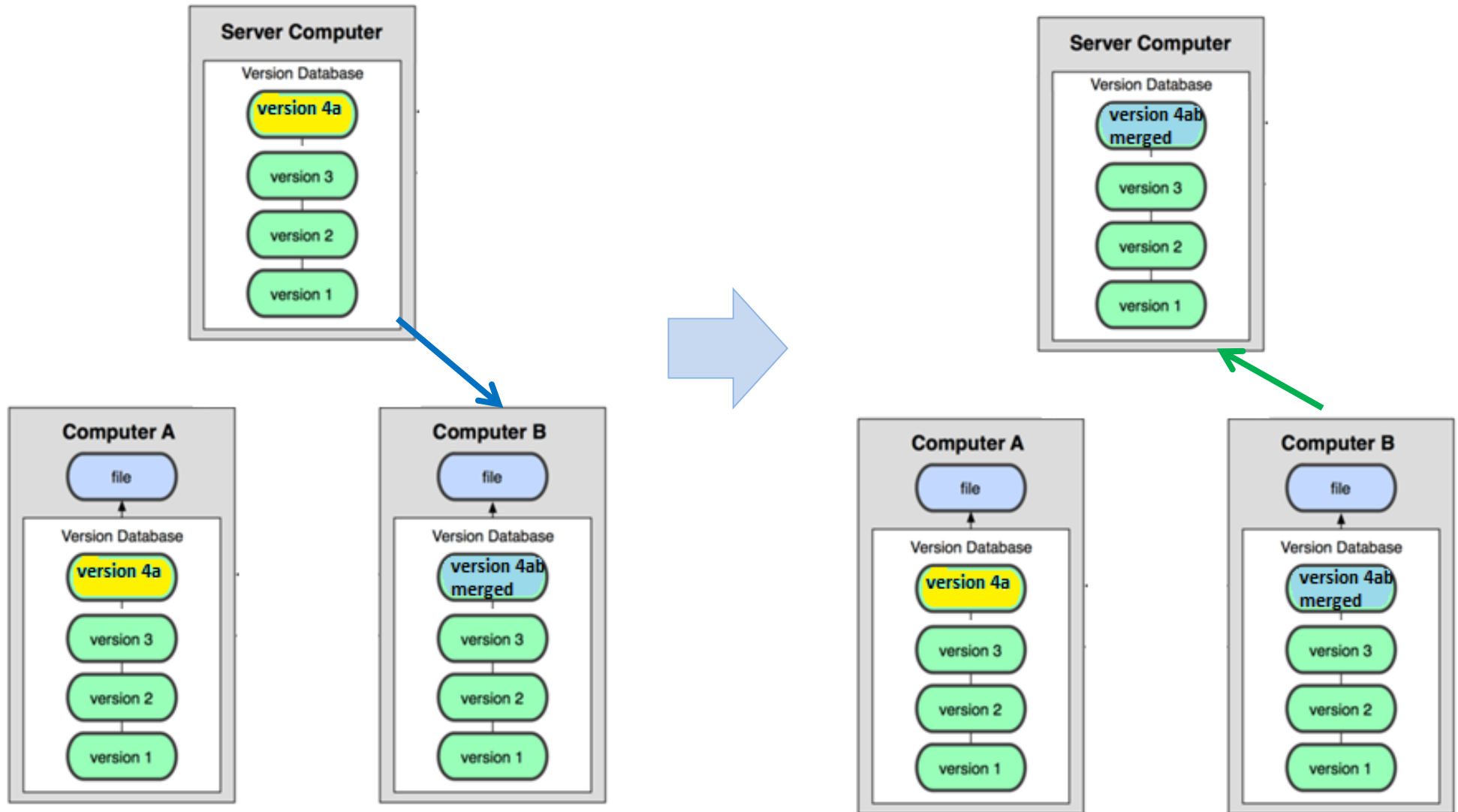
Mesclando conflitos parte 2



3. Usuário A envia os arquivos modificados ao servidor de versionamento na nuvem

4. Usuário B tenta enviar os arquivos para o servidor na nuvem mas recebe falha por conflito

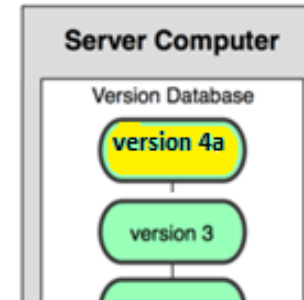
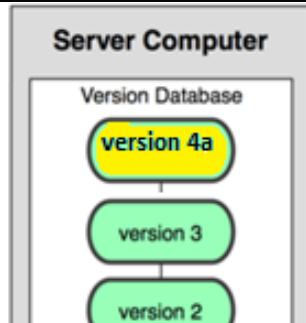
Mesclando conflitos parte 3



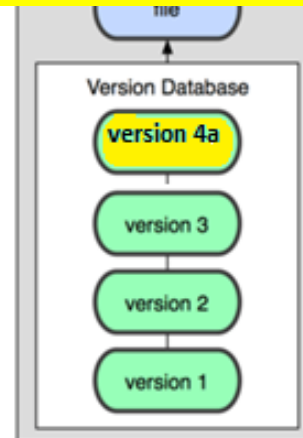
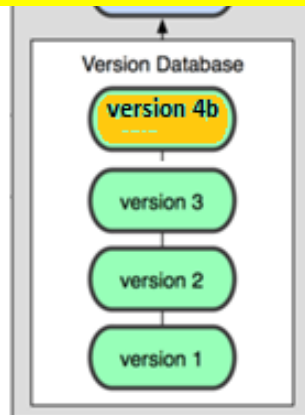
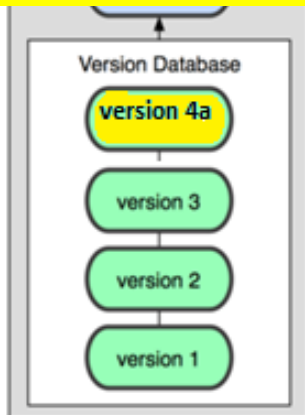
5. B recebe as alterações feitas por A e as unifica com as suas alterações automaticamente

6. Usuário B envia as suas alterações junto as do usuário A que foram reunidas em seu repositório local

Mesclando conflitos parte 4



E se A e B modificarem as mesmas partes de formas diferentes?



3. Usuário A envia os arquivos modificados ao servidor de versionamento na nuvem

4. Usuário B tenta enviar os arquivos para o servidor na nuvem mas recebe falha por conflito

Mesclando conflitos

- O arquivo conflitante conterá as seções <<< e >>> para indicar onde o Git não conseguiu resolver um conflito:

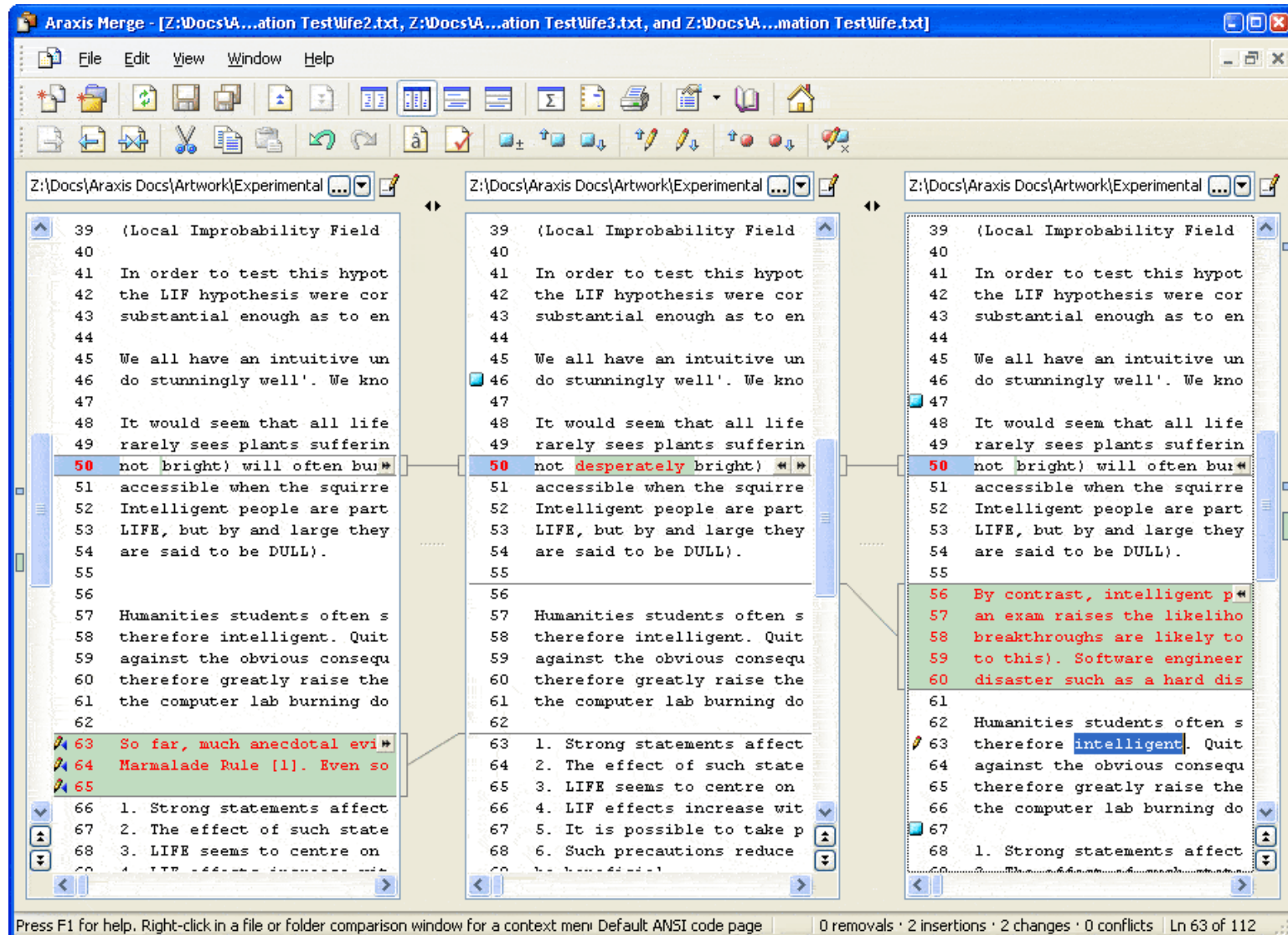
```
<<<<<<< HEAD:index.html
<div id="footer">todo: message here</div>
=====
<div id="footer">
  thanks for visiting our site
</div>
>>>>>>> SpecialBranch:index.html
```

} branch 1's version

} branch 2's version

- Encontre todas essas seções e edite-as no estado adequado (qualquer uma das duas versões é mais nova / melhor / mais correta).

Mesclando conflitos



Araxis Merge – Ferramenta para gerenciamento de conflitos de Merge

Mesclando conflitos

Workspace Version

```
public class HelloWorld {  
    public static int main(String[] args) {  
        System.out.println("Hello git world!");  
        // Change in master  
        // Change 2 in master  
        // Change 3 in master  
        // Change 4 in master  
        <<<<<< HEAD  
        =====  
        // Change 5 in master  
        >>>>>> master  
        // change 3 in release  
        return 1; // change in release  
        // change 2 in release  
    }  
  
    public static void newMethodInMaster() {  
        // change method in master  
    }  
  
    public static void newMethodInFeature1() {  
        // change method in feature 1  
    }  
}
```

Merge branch 'release/3.4.7' - f60cdec...

```
public class HelloWorld {  
    public static int main(String[] args) {  
        System.out.println("Hello git world!");  
        // Change in master  
        // Change 2 in master  
        // Change 3 in master  
        // Change 4 in master  
        // Change 5 in master  
        // change 3 in release  
        return 1; // change in release  
        // change 2 in release  
    }  
  
    public static void newMethodInMaster() {  
        // change method in master  
    }  
  
    public static void newMethodInFeature1() {  
        // change method in feature 1  
    }  
}
```

History - File: git-test/src/de/solveit/HelloWorld.java [git-test]

Id	Message	Author	Authored Date	Committer	Committed Date
f60cdec	merge branch 'release/3.4.7'	Berno Langer	2 hours ago	Berno Langer	2 hours ago
76425e0	merge branch 'release/3.4.7' into test	Berno Langer	2 hours ago	Berno Langer	2 hours ago
f92e2de	change 3 in release	Berno Langer	2 hours ago	Berno Langer	2 hours ago
cb6e90a	change 5 in master	Berno Langer	2 hours ago	Berno Langer	2 hours ago

Interagindo com o repositório remoto

- **Push** Envia suas alterações locais para o repositório remoto.
- **Pull** Puxe de repo remoto para obter as alterações mais recentes.
 - (conserte conflitos se necessário, adicione / envie-os para o seu repo local)
- Para buscar as atualizações mais recentes do repositório remoto em seu repositório local e colocá-las em seu diretório de trabalho:
 - `git pull origin master`
- Para colocar suas alterações do repositório local no repositório remoto:
 - `git push origin master`

Quem usa GIT



Servidores Git na nuvem





Página

<https://bitbucket.org>

Bitbucket



7 anos de existência

Pertence a Atlassian, dona do Trello

Integrada ao Jira, HipChat e Confluence

10 mi de usuários

28 mi de repositórios

1 mi de organizações

- Paypal, Ford



Página

<https://gitlab.com>

GitLab



8 anos de existência

Software Livre, Licença MIT

Desde 2013, GitLab Community Edition e GitLab Enterprise Edition

Usado por mais de 100k de Organizações

- Bayer, NASA, Sony e Uber
- IPqM (Marinha do Brasil), SERPRO



Página

<https://github.com>

GitHub



11 anos de existência

36 mi de usuários

2,1 mi de organizações

- Facebook, Microsoft, Airbnb, Spotify, Slack

100 mi de projetos

Microsoft compra a plataforma em 2018 (\$7,5 bi)

Atualmente é a maior e mais popular host de Git do mundo

[Pull requests](#) [Issues](#) [Marketplace](#) [Explore](#)**Microsoft**

Open source, from Microsoft with love

Redmond, WA

[https://opensource.micro](https://opensource.microsoft.com)[opensource@microsoft.c](mailto:opensource@microsoft.com) · [Verified](#)[Report abuse](#)[Repositories](#) 2.6k[Packages](#)[.NET People](#) 42k[Projects](#) 2

Pinned repositories

[VScode](#)

Visual Studio Code

[TypeScript](#)

TypeScript is a superset of JavaScript that compiles to clean JavaScript output

[.NET](#)

This repo is the official home of .NET on GitHub. It's a great starting point to find many .NET OSS projects from Microsoft, and the community, including many that are part of the .NET Foundation.

[VS Code](#) · 1.8k · 1.1k[VS Code](#) · 1.8k · 1.1k[VS Code](#) · 1.8k · 1.1k[calculator](#)

Windows Calculator: A simple yet powerful calculator that ships with Windows

[monaco-editor](#)

A browser-based code editor

[terminal](#)

The new Windows Terminal, and the original Windows console host - all in the same place

[VS Code](#) · 1.8k · 1.1k[VS Code](#) · 1.8k · 1.1k[VS Code](#) · 1.8k · 1.1k[VS Code](#) · 1.8k · 1.1k

Type: All

Language: All

Recognizers-Text

Microsoft Recognizers-Text provides recognition and resolution of numbers, units, and date/time expressed in multiple languages (CN, EN, FR, ES, PT, DE, Persian support for NLP, JA, KO). Contributions are greatly welcome! Packages are available at <https://www.nuget.org/profiles/Recognizers-Text> and <https://www.npmjs.com/~recognizers-text>

[nlp](#) [datetime](#) [parser-library](#) [timex](#)[entity-extraction](#) [nlp-entity-extraction](#) [numex](#)

Top languages

[C#](#) [TypeScript](#) [JavaScript](#)
[C++](#) [PowerShell](#)

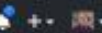
Most used topics

[azure](#) [microsoft](#) [python](#)



Search or jump to...

[Pull requests](#) [Issues](#) [Marketplace](#) [Explore](#)



Linus Torvalds

torvalds

Follow

Linux Foundation

Portland, OR

[Block or report user](#)

Organizations



Overview [Repositories 4](#) [Projects 0](#) [Stars 2](#) [Followers 99k](#) [Following 0](#)

Popular repositories

[linux](#)

Linux kernel source tree

79k 27.5k

[uemacs](#)

Random version of microemacs with my private modifications

414 63

[test-tlb](#)

Stupid memory latency and TLB tester

227 72

[pesconvert](#)

Another PES file converter

117 12

[subsurface-for-dirk](#)

Forked from Subsurface-devel:subsurface

Do not use - the real upstream is Subsurface-devel:subsurface

92 22

[libdc-for-dirk](#)

Forked from Subsurface-devel:libdc

Only use for syncing with Dirk, don't use for anything else

46 19

2,359 contributions in the last year



Contribution activity

August 2019

2019

2018

On a beautiful April day on GitHub...

80k

Repositories
Updated

7k

People pushed
their first repository

4k

People forked
a repository
for the first time

3k

People created
their very first
pull request

30k

Issues created

12k

Pull Requests
created

And **6k** people signed up to join the fun.



GitHub



Relatório de atividades de um único dia no
GitHub, feito em maio de 2012.

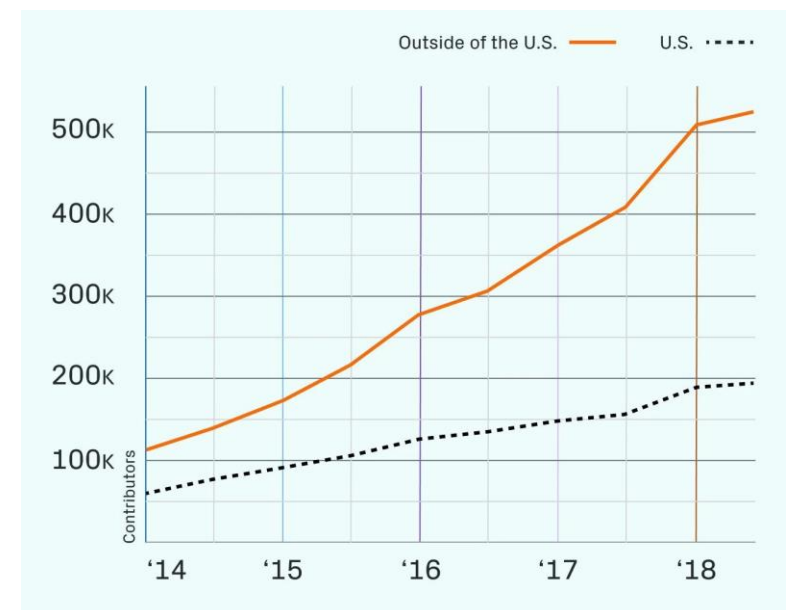
Página

<https://github.blog/2012-05-01-data-at-github/com>

GitHub



Onde esses repositórios são criados

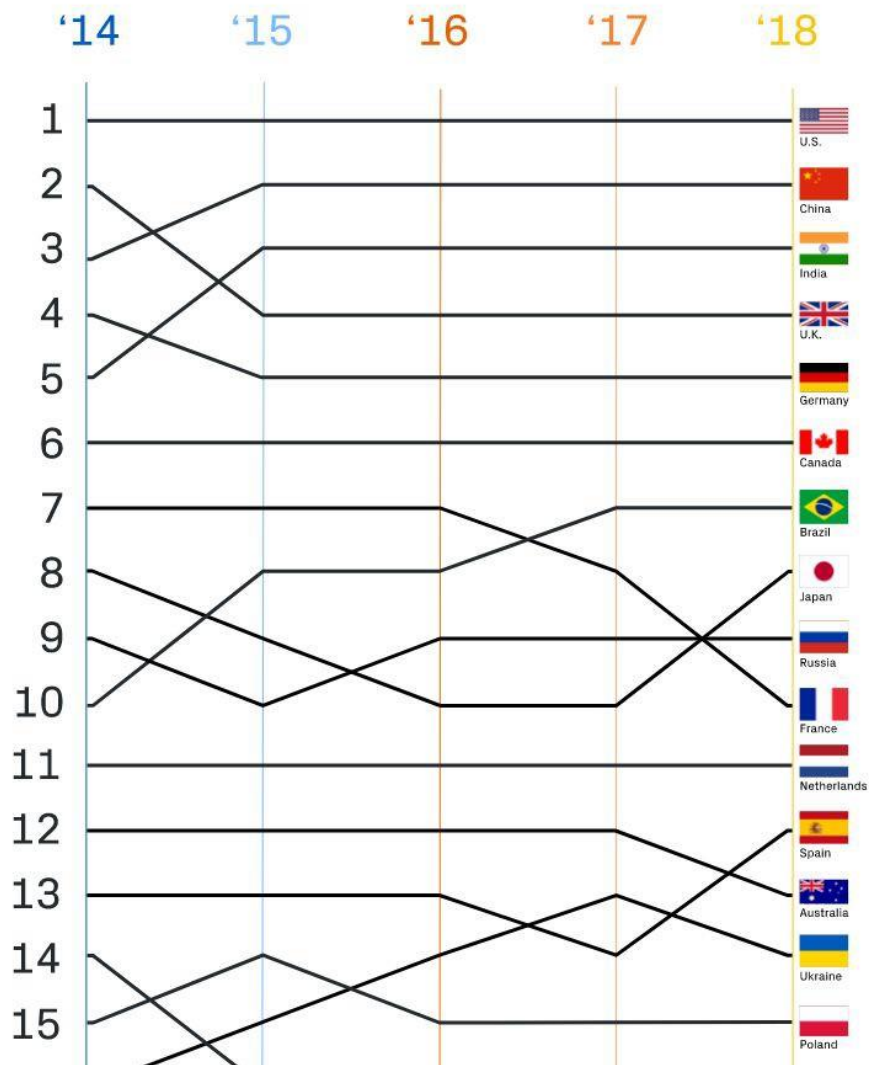


	Contributors
1 Microsoft/vscode	19k
2 facebook/react-native	10k
3 tensorflow/tensorflow	9.3k
4 angular/angular-cli	8.8k
5 MicrosoftDocs/azure-docs	7.8k
6 angular/angular	7.6k
7 ansible/ansible	7.5k
8 kubernetes/kubernetes	6.5k
9 npm/npm	6.1k
10 DefinitelyTyped/DefinitelyTyped	6.0k



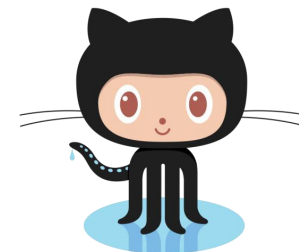
Página

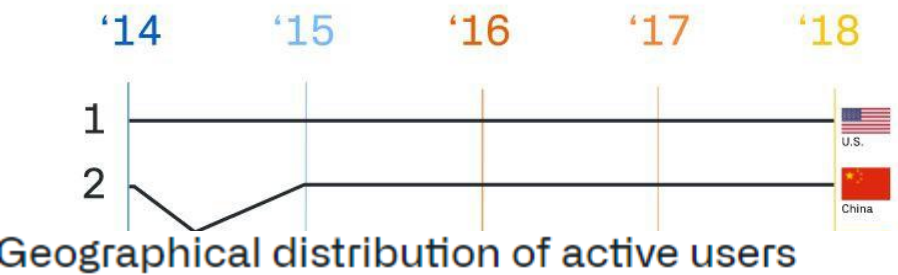
<https://github.blog/2018-11-08-100m-repos/>



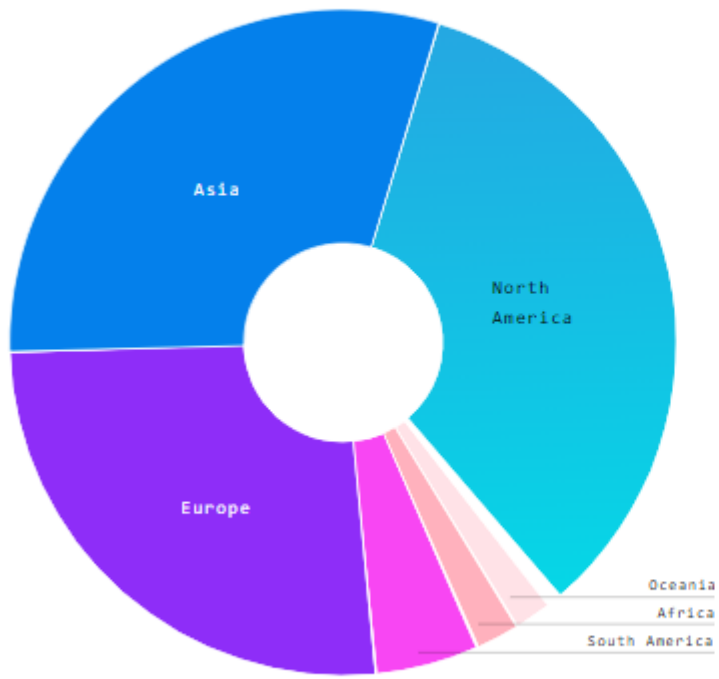
GitHub

Mais algumas informações interessantes...

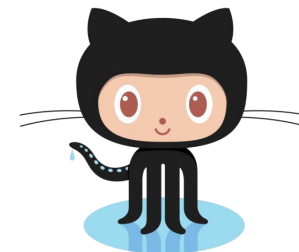




Dados de 09/2021



GitHub



34.8%

North America

Decrease of 2% from last year

30.7%

Asia

Increase of 1.1% from last year

26.8%

Europe

Increase of 0.1% from last year

4.9%

South America

Increase of 0.4% from last year

2.0%

Africa

Increase of 0.3% from last year

1.7%

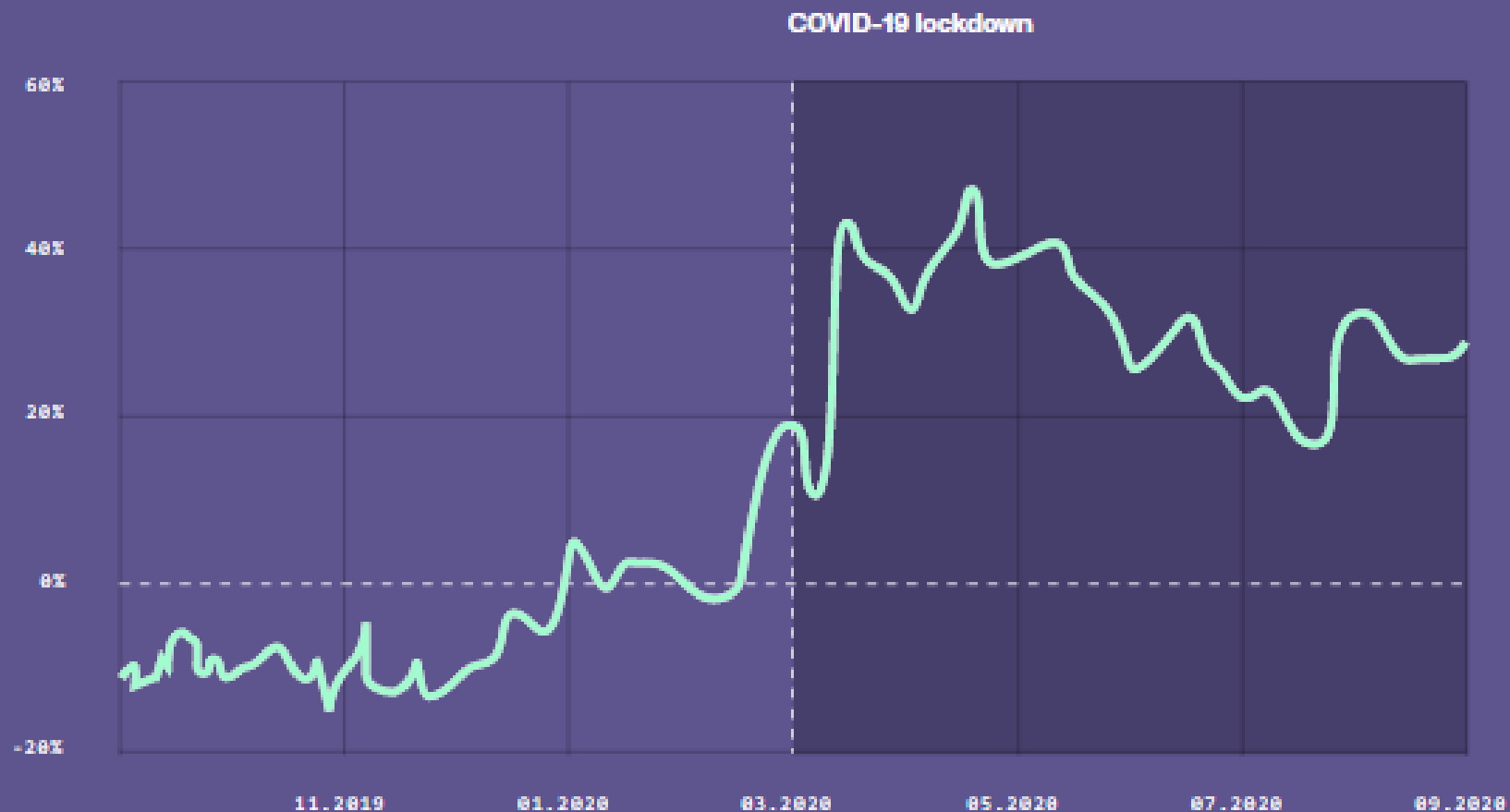
Oceania

No change since last year



Percent increase in open source project creation per active user compared to previous year

Seven day rolling average



GitHub



Mais algumas informações interessantes...

Based on the data collection range of October 2019 - September 2020.

56 M+
total developers on
GitHub

60 M+
new repositories created
in the last year

72 %
of Fortune 50 companies
use GitHub Enterprise

1.9 B+
contributions added
in the last year

Servidores na nuvem

- *Pergunta: Eu sempre tenho que usar o servidores na nuvem para o Git?*
 - *Resposta:* Não! Você pode usar o Git localmente para seus próprios propósitos.
 - Ou você ou outra pessoa poderia configurar um servidor para compartilhar arquivos.
 - Ou você pode compartilhar um repo com os usuários no mesmo sistema de arquivos, desde que todos tenham as permissões de arquivo necessárias.

Ferramentas gráficas para GIT

- → GitEye
 - <https://www.collab.net/products/giteye>
- → Smartgit
 - <https://www.syntevo.com/smartgit/>
- → Tig
 - <https://jonas.github.io/tig/>
- → Gitcola
 - <http://git-cola.github.io/>
- → Mais Opções
 - <https://git-scm.com/downloads/guis/>

É útil só para código?

- É útil apenas para código?
 - Não, é útil para
 - Documentos em Latex
 - Documentos em HTML
 - Versionamento de imagens
 - Documentos do Word (mas não perfeitamente [entenda aqui](#))
 - Linguagens de especificação de circuitos cujos arquivos sejam textuais compactados ou não
 - Documentação em geral.
- E se utilizo Word
 - [O Word tem versionamento embutido](#). Mas só garante histórico e diff, nada de ramificações ou controle de acesso
 - Soluções alternativas
 - Word online (limitadas ferramentas de edição)
 - Google Docs (interface distinta ferramentas limitadas)
 - Dropbox (sem acesso simultâneo, mas com controle de histórico e diff)
 - [Simuldocs](#) (Git para Word 100% funcional grátis até 25 docs)